GNN Guided Mapping of Data Flow Graphs for Coarse-Grained Reconfigurable Arrays

Capstone Project Final Report

Deniz Lopes Günes

Bachelor's in Informatics and Computing Engineerin

U.Porto's Tutor: Professor João Cardoso
Company's Tutor: Professor Nuno Paulino

June 2026

Contents

Glossary

AI *Artificial Intelligence*; field of computing dedicated to techniques that enable systems with adaptive or learned behaviour.

ALAP *As Late As Possible*; classical scheduling priority that schedules each operation at the latest cycle compatible with its data dependencies.

ALU *Arithmetic Logic Unit*; combinational unit inside a PE that performs the arithmetic and logical operations of the supported opcodes.

ASAP *As Soon As Possible*; classical scheduling priority that schedules each operation at the earliest cycle compatible with its data dependencies.

CGRA *Coarse-Grained Reconfigurable Architecture*; reconfigurable architecture composed of an array of configurable processing units.

CGRA-ME *CGRA Modelling and Exploration*; open-source CGRA design framework used as the architecture template by the modified SAT-MapIt in this project.

DAG *Directed Acyclic Graph*; directed graph with no cycles, used here as the underlying structure of a DFG.

DFG *Data Flow Graph*; directed graph that represents operations as nodes and data dependencies as edges.

DQN *Deep Q-Network*; reinforcement learning agent that approximates the Q-function with a deep neural network.

FEUP *Faculdade de Engenharia da Universidade do Porto*; Faculty of Engineering of the University of Porto.

GNN *Graph Neural Network*; neural network designed to process data structured as graphs.

INESC TEC *Instituto de Engenharia de Sistemas e Computadores, Tecnologia e Ciência*; Institute for Systems and Computer Engineering, Technology and Science.

JSON *JavaScript Object Notation*; text-based format used to store and exchange structured data.

KMS *Kernel Mobility Schedule*; modulo-scheduling construct that bounds the cycles in which each operation may be issued, used by SAT-MapIt [7] to reduce the SAT search space.

L.EEC *Licenciatura em Engenharia Eletrotécnica e de Computadores*; Bachelor's Degree in Electrical and Computer Engineering at FEUP.

label0 Label predicted by the LISA-GNN and used as a priority in the greedy partitioner.

LISA *Graph Neural Network Based Portable Mapping on Spatial Accelerators* [5]; GNN-based approach used to guide mapping on spatial accelerators.

LISA-GNN GNN module used in the prototype, inspired by the LISA [5] approach, to predict priorities for DFG nodes.

LLVM Compiler infrastructure project providing modular and reusable compiler and toolchain technologies, used by the reference LISA flow.

Mapper Component that searches for or verifies a mapping solution.

Mapping Process of assigning the operations of a DFG to the physical resources of the CGRA.

NaviMap GNN plus deep-Q-network mapping approach for CGRAs, considered and ultimately discarded as a third leg of the prototype.

One-shot Model in which a partition of the DFG is mapped onto a single spatial configuration of the CGRA.

PE *Processing Element*; individual processing unit within the CGRA.

SAT *Boolean Satisfiability*; problem of determining whether an assignment exists that satisfies a set of Boolean constraints.

SAT-MapIt SAT-based mapper for CGRAs [7], used as a technical reference in this project.

SDG *Sustainable Development Goals*; United Nations Sustainable Development Goals.

SMT *Satisfiability Modulo Theories*; generalisation of SAT with theories such as arithmetic, arrays and bit-vectors.

Z3 SMT/SAT solver [1] used to discharge the mapping constraints in the prototype.

1 Introduction

1.1 Context

This report documents the work carried out in the Capstone Project course unit, part of the Bachelor's Degree in Informatics and Computing Engineering at the Faculty of Engineering of the University of Porto (FEUP), during the second semester of the 2025/2026 academic year.

The project was developed at **INESC TEC**, under the supervision of Professor Nuno Paulino and the tutorship of Professor João Cardoso. The goal was to investigate and implement an approach inspired by LISA[5], based on *Graph Neural Networks* (GNNs), for the *mapping* of *Data Flow Graphs* (DFGs) onto *Coarse-Grained Reconfigurable Architectures* (CGRAs).

INESC TEC is an associate laboratory active in engineering and computer science, hosting applied research in compilation and reconfigurable architectures, which are the areas in which this project is positioned.

1.2 Goals and expected results

The goals defined for the project were the following:

- Study the state of the art on *mapping* of DFGs to CGRAs, in particular GNN-based approaches inspired by LISA.
- Build a prototype that departs from current approaches by using GNNs to guide *mapping* decisions.
- Evaluate the solution with end-to-end examples and contrast it with a purely algorithmic strategy.

1.3 Report structure

The next section frames the methodology, stakeholders and activities. The development section describes the DFG model, the CGRA model, the partitioning, the materialisation of dependencies between partitions, the SAT validation and the integration of the LISA-GNN. The experimental validation, known limitations and future work follow.

2 Methodology and main activities

2.1 Methodology

This project was conducted at INESC TEC in a hybrid model. I had the flexibility to work on the project remotely and to communicate with my supervisors and the rest of the team through a Slack[6] workspace that belongs to the team and to which I was invited.

Throughout the process, weekly meetings were held in the INESC TEC room in which I presented what had been accomplished during the week and we discussed the results and what should be addressed in the following one. For these meetings I prepared a short presentation of about 5-10 minutes. These meetings were an opportunity not only to clarify questions about the project and its implementation, but also to receive feedback from the supervisors regarding the current state of the prototype and the features to be integrated next. This is also why the project requirements evolved significantly over the semester: although the main objective was clear (use *AI* to guide the mapping process), the structure of the pipeline and exactly which state of the art approaches to follow were not fixed at the start, and the meetings were essential to decide which ideas were appropriate given the project's goals and duration.

The development overall followed an iterative methodology[2], well suited to a research project of this nature. The project was developed inside a private GitLab[3] repository belonging to the team.

The technical resources used were:

- GitLab[3] for version control.
- Slack[6] for day-to-day communication with the team.

2.2 Stakeholders, roles and responsibilities

The project was developed individually by Deniz Lopes Günes.

- **Deniz Lopes Günes (author)**: literature review, implementation of the DFG generator, the partitioner, the integration of the SAT solver, the adaptation of SAT-MapIt, the training and inference of the LISA-GNN, the end-to-end examples and the writing of the report.
- **Francisco Mouro Sampaio Nunes (team colleague, L.EEC)**: developed a project vertical to mine, responsible for generating the code used to configure the *CGRA* from the *mappings* produced.
- **Professor Nuno Paulino (main supervisor, INESC TEC)**: technical guidance, definition of goals, critical review of the work and validation of the results.
- **Guilherme Cruz (co-supervisor, INESC TEC)**: technical support and discussion of implementation strategies.
- **Henrique Teixeira (co-supervisor, INESC TEC)**: technical support and discussion of implementation strategies.

- **Professor João Cardoso (tutor, FEUP)**: academic accompaniment, alignment with the course unit's requirements and critical review of the report.

2.3 Activities carried out

The technical activities can be grouped into the following blocks:

- **DFG representation**: definition of a JSON format for nodes, operations, constants, metadata and dependency edges.
- **Example generation**: creation of synthetic graphs inspired by matrix multiplication and convolution patterns.
- **Partitioning**: development of strategies to split graphs that do not fit in a single CGRA configuration.
- **Materialisation**: conversion of inter-partition dependencies into synthetic *load*/*store* nodes.
- ***Mapping* validation**: SAT verification of each materialised partition, respecting operation compatibility, connectivity between PEs and resource limits.
- **LISA-GNN integration**: use of *label0* predictions as a priority for node selection in the greedy partitioner.
- **Evaluation**: execution of end-to-end examples and collection of metrics on partitioning, cuts and SAT validation.

The initial phases (study, DFG generation, partitioning) took place between the start of the semester and mid-March; the LISA-GNN integration and the end-to-end pipeline were consolidated in May; the evaluation and report writing followed in the final weeks.

The remainder of this section walks through the project's evolution meeting by meeting, which is the most faithful way to describe how the direction of the work was iteratively reshaped.

2.3.1 First meeting (20 February 2026)

The first meeting was used to re-introduce the team, agree on the communication channel (Slack) and align on the immediate reading material. I was tasked with surveying the state of the art on graph *mapping* for CGRAs, in particular GNN-based approaches, so that the next meeting could be grounded on concrete papers rather than on the generic proposal.

2.3.2 Second meeting (27 February 2026)

After the initial reading pass, the supervisors asked me to bring a dataset of DFGs of the kind used in the state-of-the-art papers, to review two or three of the most cited mappers, and to try out a conventional mapper as a baseline. The discussion also opened two questions that would stay with the project until the end: how to check that a *mapping* is valid, and how to represent a CGRA as a graph. The recommendation was to keep both representations as simple as possible.

2.3.3 Third meeting (6 March 2026)

By the third meeting I had cloned the LISA[5] DFG generator and produced a first synthetic dataset of graphs. Reproducing the LISA flow end-to-end ran into a practical blocker: LISA itself relies on *Gurobi* on top of an *LLVM* build, and the *LLVM* dependency could not be installed cleanly in the working environment. In parallel, I started reading SAT-MapIt[7], which uses a *Kernel Mobility Schedule* (KMS), a construct from *modulo scheduling* that bounds the cycles in which each operation may be issued, to reduce the SAT search space. The key idea that came out of this meeting was the one that ended up defining the whole project: rather than picking one of the two approaches, combine them. Use the LISA-style GNN to add constraints to the graph, and let a SAT-based mapper consume those constraints.

2.3.4 Fourth meeting (20 March 2026)

Two things were settled in this meeting. First, the LISA flow was reproduced for the homogeneous case: 1000 DFGs were generated for a 4×4 CGRA and another 1000 for an 8×8 CGRA, although mapping all of them with the reference flow turned out to be impractical (roughly 40 graphs mapped in a full day of runtime for the 4×4 case). Second, I started reading *NaviMap*, a GNN-plus-*deep-Q-network* mapping approach, and we discussed whether the LISA labels could be fed into a NaviMap-like agent. The idea was left open as a possible third leg of the prototype, to be evaluated in the following meeting after a deeper read of the paper.

2.3.5 Fifth meeting (27 March 2026)

This meeting was when the NaviMap direction was dropped. After a closer read, combining a *deep-Q-network* agent on top of the GNN labels turned out to add significant implementation and training complexity, having to handle different input shapes for the Neural Networks as well as a lack of open-source availability from NaviMap, without a clear advantage over the simpler LISA-plus-SAT-MapIt pairing for the scale of graphs we were targeting. The decision was therefore to set NaviMap aside and to concentrate the remaining effort on the LISA-style labels combined with the SAT mapper.

2.3.6 Sixth meeting (10 April 2026)

The discussion in this meeting was almost entirely about the *codegen* side of the project, led by Francisco. From my side, the main takeaway was that the mapping flow had to produce a clean, stable JSON output, so that the downstream code generation could consume it without further negotiation about the format.

2.3.7 Seventh meeting (17 April 2026)

At this point I had a Python `CGRA` class that is passed to the mapper, LISA was producing four labels per node (used as features), and the modified SAT-MapIt was producing ground-truth mappings using two of those labels (temporal and spatial distance) on top of a *CGRA-ME*-style template. The meeting surfaced a real, recurring problem: the DFG formats used by LISA and by SAT-MapIt did not agree. LISA worked with a small set of operations (`const`, `load`, `store`, `add`, `mul`, `sub`, ...), whereas SAT-MapIt also reasoned about branches, *live-ins/live-outs* and other entities. The action item was to consolidate a single input format and intermediate JSON, under the assumption that everything is one-shot, and to check whether SAT-MapIt could be coerced into a one-shot regime. This meeting was when we decided to simplify the problem by focusing on one-shot mapping with no temporal constraints.

2.3.8 Eighth meeting (24 April 2026)

This is the meeting in which the eventual shape of the prototype was decided. The problem on the table was that SAT-MapIt is built around *modulo scheduling*, its notion of an *initiation interval*, and the way it places operations across cycles all assume that regime, whereas LISA is essentially agnostic to it and produces per-node features without committing to a schedule. Plugging the LISA labels straight into SAT-MapIt therefore did not collapse cleanly into the one-shot model we wanted. My suggestion was to invert the approach: take a large DFG, run a learned model on it to obtain per-node features, use those features to *split* the graph in such a way that the cut introduces synthetic loads and stores, and then one-shot *map* each subgraph with the SAT mapper. The open question became how to perform the split itself, whether by a heuristic or by another learned component.

2.3.9 Ninth meeting (30 April 2026)

The ninth meeting confirmed that the partitioned one-shot strategy worked. A greedy partitioner was already in place, not yet driven by a GNN, but by hand-crafted priorities such as *ASAP*, *ALAP* and edge count — whose objective was to minimise the number of edges cut at the partition boundary, and thus the number of synthetic loads and stores needed to keep memory operations outside of the CGRA's ALU array. The splitting was working and each subgraph was being mapped one-shot with the modified SAT-MapIt to produce ground truth.

From here the plan was clean: use the original large graph, augmented with the per-partition timesteps as an extra feature, to train a GNN that predicts the schedule order (*label0*); then feed *label0* into the partitioner to obtain better partitions, and finally let SAT-MapIt map each partition. This is the flow that ended up being reported in this document.

3 Development of the solution

3.1 Requirements

The project was always very open-ended in terms of the technical solution, as long as it iterated on state of the art approaches using *AI*. One technical constraint was to agree on a JSON format for the output since another project for the codegen part was being developed in parallel, and the mapping output had to be clean and stable for that project to consume it without further negotiation.

The prototype does not cover cycle-by-cycle temporal scheduling. The SAT *mapper* only verifies the spatial validity of each partition, that is, it assumes that internal dependencies are resolved by direct connections between PEs in the same configuration.

3.2 Architecture and technologies

The solution was implemented in Python and organised as a modular pipeline:

$$\texttt{DFG generation} \rightarrow \texttt{LISA-GNN inference} \rightarrow \texttt{partitioning} \rightarrow \texttt{materialisation} \rightarrow \texttt{SAT mapping}$$

The generator creates synthetic data flow graphs. The LISA-GNN inference module computes predictions for the graph nodes. The partitioner receives the DFG, the CGRA description and, optionally, those predictions. The result is an ordered sequence of partitions, each one converted into a materialised DFG. Finally, each partition is verified by a SAT mapper implemented with Z3[1] and exported as a PE configuration.

3.2.1 DFG model

A DFG is modelled as a directed acyclic graph (DAG) in which the nodes represent operations and the edges represent data dependencies. Constants are root nodes and memory operations are represented explicitly through `load` and `store`. In the current model, all edges have zero distance, that is, a dependency inside a partition must be satisfied in the same one-shot configuration.

3.2.2 CGRA model

The CGRA is described by a grid of PEs. Each PE can support a subset of operations and, in some architectures, only some PEs can perform memory operations. The topology defines which PEs are directly connected. In the current experiments a homogeneous 4×4 CGRA with five registers per PE was used. Figure 1 shows the canonical layout we assume: a 2D grid of PEs, with data and instruction memories on the side, where each PE is essentially an ALU with a local register file and can operate on the results delivered by its neighbouring PEs.

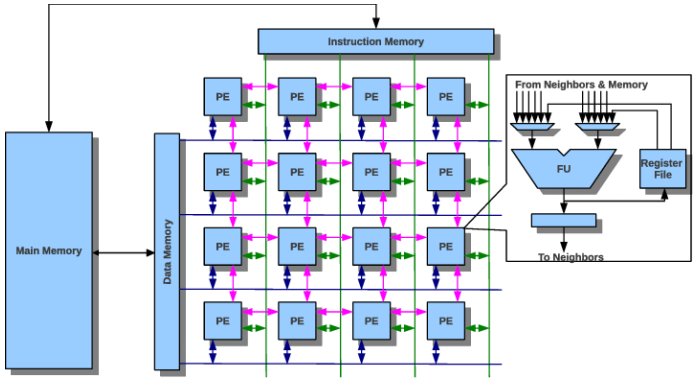

Figure 1: A 4×4 CGRA is essentially a 2D grid of PEs, plus data and instruction memories; each PE can operate on the results of its neighbouring PEs, is built around an ALU with a local register file, and shares a global storage area in the data memory reserved by the compiler. Figure reproduced from Jeyapaul *et al.*[4].

3.2.3 One-shot mapping validity

For a partition to be considered mappable, the SAT solver must find an assignment of nodes to PEs that satisfies the following conditions:

Let N be the set of instruction nodes of the partition, E the set of instruction-to-instruction edges, and P the set of PEs of the CGRA. For every $(n, e) \in N \times P$ we introduce a Boolean placement variable

$$p_{n,e} \in \{0, 1\}, \qquad p_{n,e} = 1 \iff \text{node } n \text{ is placed on PE } e.$$

The mapping is then encoded by the following clauses:

(C1) Operation compatibility. For each node n and each PE e that does not support the opcode of n (denoted $e \notin ops(n)$), the placement is forbidden:

$$\forall n \in N,\ \forall e \in P,\ e \notin ops(n) \implies \neg p_{n,e}. \tag{1}$$

(C2) Exactly one PE per instruction. Each instruction node is placed on exactly one PE, enforced as a pseudo-Boolean equality:

$$\forall n \in N,\ \sum_{e \in P} p_{n,e} = 1. \tag{2}$$

(C3) At most one instruction per PE. Each PE may host at most one instruction of the partition:

$$\forall e \in P,\ \sum_{n \in N} p_{n,e} \leq 1. \tag{3}$$

(C4) Same-cycle nearest-neighbour dependencies. Let $nbr(a, b)$ denote that PEs a and b are directly connected in the CGRA topology (or that $a = b$ when the model allows intra-PE forwarding). For every internal dependency $u \to v \in E$, at least one feasible neighbouring assignment must be selected:

$$\forall (u, v) \in E,\quad \bigvee_{\substack{a,b \in P \\ nbr(a,b)}} \big(p_{u,a} \wedge p_{v,b}\big). \tag{4}$$

This is a simplification of SAT-MapIt's temporal forward-edge constraint: dependent instructions may share the same partition (and therefore the same one-shot cycle) as long as their PEs are physically adjacent.

(C5) Memory-capable PEs. For each memory node n (`load` or `store`) and each PE e that does not provide memory access ($e \notin mem$):

$$\forall n \in N_{mem},\ \forall e \in P,\ e \notin mem \implies \neg p_{n,e}. \tag{5}$$

In the current experiments we assume a homogeneous CGRA in which every PE supports every operation and every PE has memory access, so $mem = P$ and (C5) is always satisfied;

Post-solution resource checks. Fan-in, fan-out and physical-link capacity are not encoded in the SAT formulation. Once Z3 returns a model, the prototype reads back the placements and rejects any solution in which, for the chosen limits, a PE exceeds its outgoing or incoming edge budget, or a physical link between two PEs is used by more edges than its capacity. A violation here is reported back to the caller as an infeasible mapping rather than as a SAT failure.

A structure such as $A \to C \leftarrow B$ is valid within a single partition if A, B and C are placed on compatible PEs and if the PEs of A and B can deliver their results directly to the PE of C, within the limits of C's inputs and the physical links. It should be noted that the prototype covers spatial feasibility inside the partition, not temporal scheduling.

3.2.4 DFG partitioning

When the full DFG does not fit in a single CGRA configuration, the graph is split into a sequence of partitions $P_1, \ldots, P_K$. To preserve the order of the dependencies, the partitioner enforces:

$$partition(src) \leq partition(dest) \tag{6}$$

for each edge $src \rightarrow dest$. This constraint prevents dependencies that would have to flow back to a previous partition.

Two strategies were implemented:

- **sat-min-k**: searches for the smallest feasible number of partitions, starting from a lower resource bound and validating each candidate via SAT;
- **greedy**: builds a sequence of partitions through a greedy pass over the ready nodes, optionally using priorities provided by the LISA-GNN.

3.2.5 Materialisation of inter-partition dependencies

An edge that crosses a partition boundary cannot be represented as a direct connection between PEs, because the two operations belong to distinct configurations. Therefore, each cut edge $u \rightarrow v$ is materialised as:

```
P_i:  u -> synthetic_store        P_j:  synthetic_load -> v
```

The two synthetic nodes share the same logical memory reference. The overall approach is therefore a **partitioned one-shot mapping**: temporal decomposition between partitions, spatial *mapping* within each partition.

3.2.6 LISA-GNN integration

The LISA-GNN integration follows the motivation of LISA[5]: leveraging learned information about the DFG structure to guide the *mapping*. In the prototype, the GNN predicts a per-node priority *label0*, which defines the exploration order of the ready nodes in the greedy partitioner. The GNN does not replace the solver: the validity of the *mapping* still depends on the structural constraints and the SAT verification.

3.3 Developed solution

The developed solution is a Python pipeline that takes a DFG and a CGRA description as input and produces, for each one-shot partition of the graph, a SAT-validated placement of the partition's instructions onto the PEs of the CGRA. The pipeline is organised in five stages, each implemented as an independent module and exercised by the same set of end-to-end tests:

1. **DFG generation.** Either a synthetic graph is produced by the in-tree generator (`dfg_gen`), or an existing JSON DFG is loaded from disk. The format follows the model described in Section 3.2.1 — instruction nodes, constant nodes and `load`/`store` memory nodes, with zero-distance data dependency edges.

2. **LISA-GNN inference (optional).** When enabled, the LISA-GNN module reads the DFG and produces a per-node *label0* prediction, used downstream as a priority by the greedy partitioner.

3. **Partitioning.** The DFG is split into an ordered sequence $P_1, \ldots, P_K$ of subgraphs that respect the topological order of the original graph. Two strategies are available: `sat-min-k`, which probes increasing values of K and validates each candidate via SAT, and the *greedy* variant, optionally driven by the LISA-GNN priorities.

4. **Materialisation.** Each edge that crosses a partition boundary is rewritten as a pair of synthetic `store`/`load` nodes that share a logical memory reference, yielding one self-contained DFG per partition.

5. **SAT mapping.** Each materialised partition is handed to the SAT mapper described in Section 3.2.3, which either returns a valid placement or rejects the partition.

The resulting artefacts are a sequence of per-partition JSON mappings, each one listing the PE assignment of every node together with the operand wiring (PE-to-PE links, immediate constants and memory references). This is the input consumed downstream by the code generation work developed in parallel by the team colleague.

Two end-to-end examples are bundled with the prototype as smoke tests and reference inputs: a synthetic matrix multiplication kernel and a synthetic convolution kernel. Both fit in two one-shot partitions on a homogeneous 4×4 CGRA and exercise the full pipeline from DFG generation through SAT mapping. They are also the inputs used in the validation reported in the next subsection. Figures 2 to 7 show, for each example, the original DFG, the partitioning produced by the pipeline, and the resulting placement on the 4×4 CGRA.

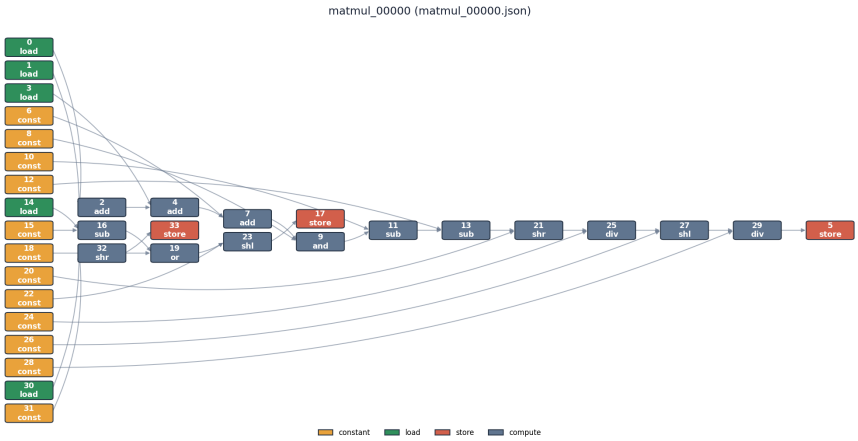

Figure 2: Synthetic DFG of matrix multiplication.

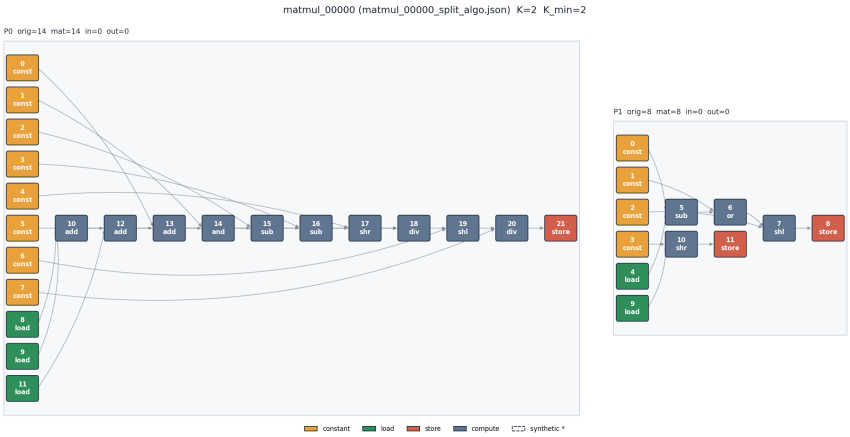

Figure 3: Partitioning of the matrix multiplication DFG into one-shot partitions.

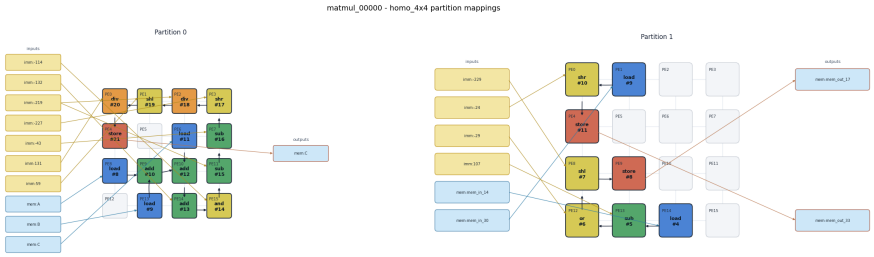

Figure 4: Mapping of the matrix multiplication partitions onto a 4×4 CGRA.

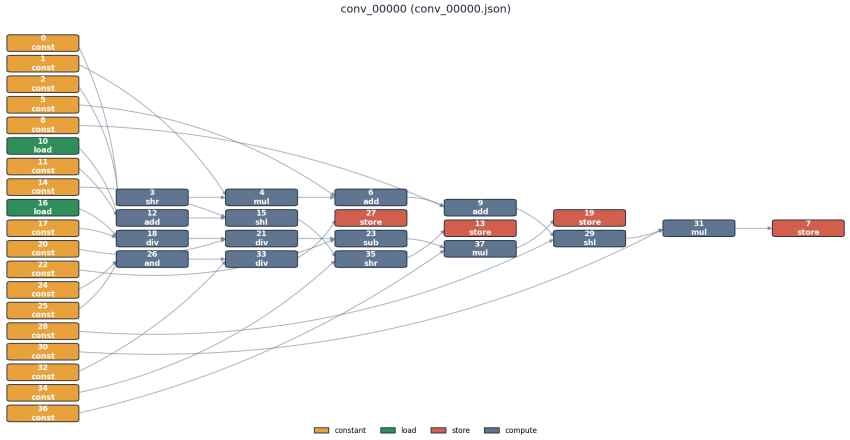

Figure 5: Synthetic convolution DFG.

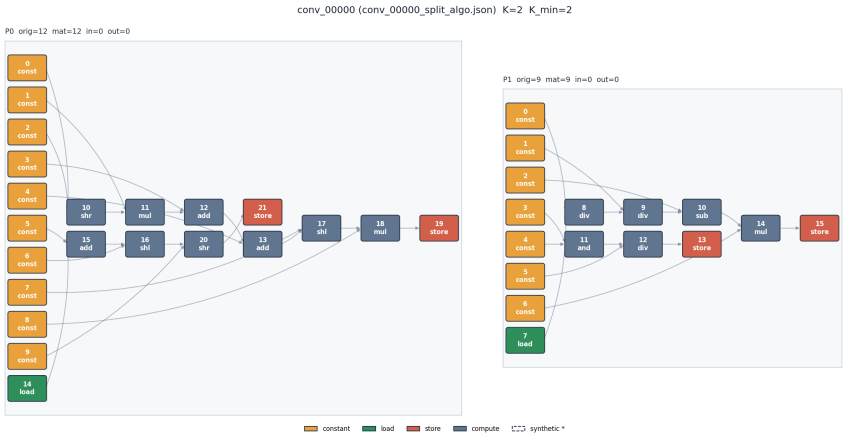

Figure 6: Partitioning of the convolution DFG into one-shot partitions.

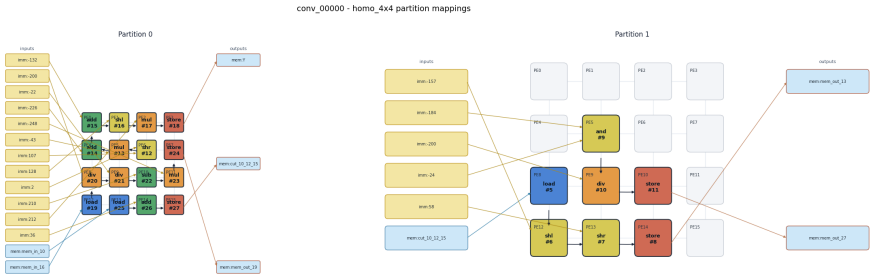

Figure 7: Mapping of the convolution partitions onto a 4×4 CGRA.

3.4 Validation

4 Conclusions

4.1 Results achieved

4.2 Lessons learned

4.3 Future work

4.4 SDG impact

The project aligns with SDG 9, Industry, Innovation and Infrastructure, by investigating compilation and optimisation techniques for reconfigurable architectures. More efficient mappings for CGRAs can reduce the energy consumption of computational workloads accelerated on this type of hardware, even though that gain was not quantified in this work.

References

[1] Leonardo de Moura and Nikolaj Bjørner. Z3: An efficient smt solver. In *Tools and Algorithms for the Construction and Analysis of Systems*, volume 4963 of *Lecture Notes in Computer Science*, pages 337–340. Springer, 2008.

[2] Mihai Liviu Despa. Comparative study on software development methodologies. *Database Systems Journal*, 5(3):37–56, 2014.

[3] GitLab Inc. Gitlab. `https://gitlab.inesctec.pt/`, 2026. [Online; accessed 18 May 2026].

[4] Reiley Jeyapaul, Shrikant Marathe, and Aviral Shrivastava. Enabling multithreading on CGRAs. In *2011 International Conference on Parallel Processing*, pages 255–264. IEEE, 2011. Figure 1 reproduced with credit; accessed via ResearchGate, 19 May 2026.

[5] Zhaoying Li, Dan Wu, Dhananjaya Wijerathne, and Tulika Mitra. LISA: Graph neural network based portable mapping on spatial accelerators. In *2022 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, pages 444–459. IEEE, 2022.

[6] Slack Technologies. Slack. `https://slack.com/`, 2026. [Online; accessed 18 May 2026].

[7] Cristian Tirelli, Lorenzo Ferretti, and Laura Pozzi. SAT-MapIt: A sat-based modulo scheduling mapper for coarse grain reconfigurable architectures. In *2023 Design, Automation & Test in Europe Conference & Exhibition (DATE)*, pages 1314–1319. IEEE, 2023.